SymBoltz.jl: A symbolic-numeric, approximation-free & differentiable Boltzmann code

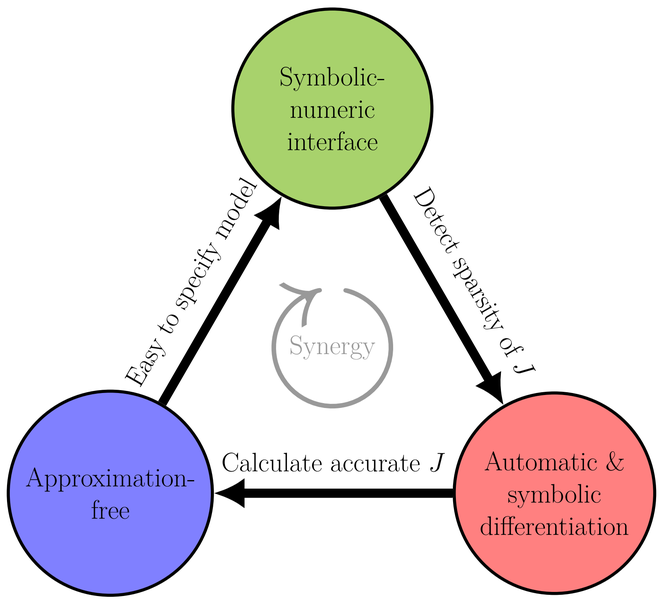
Herman Sletmoen

UNIVERSITY OF OSLO

github.com/hersle/SymBoltz.jl

2509.24740

Main features and goals



- **Symbolic-numeric:** Users enter symbolic equations → compiled to numerical code.
 - Easier to modify eqs. than in CAMB & CLASS. No need to dive into low-level code.
 - Automates repetitive tasks, such as separation/splining of background/perturbations.
 - Extra info to ODE solver, e.g. anal. Jacobian that is unknown in CAMB & CLASS.
- **Approximation-free:** Integrates full equations at all times with stiff ODE solvers.
 - No TCA, UFA, RSA or NCDMFA, which significantly complicate CAMB & CLASS.
 - Much simpler: no need to rederive and recheck approximations for model extensions.
- **Differentiable:** Compute $\partial(\text{output})/\partial(\text{input})$ with automatic differentiation.
 - For example: $\partial P(k)/\partial\Omega_{m0}$, $\partial C_\ell^{\mathrm{TT}}/\partial h$ or $\partial d_L(z)/\partial\Omega_{\Lambda 0}$.
 - Uses numerically exact chain rules. No tuning of ϵ with finite differences.
 - Useful for Fisher forecasting & gradient-based MCMC (HMC) and optimization.

Simplicity: SymBoltz vs. CLASS

Massless neutrino equations in SymBoltz:

```
D(Fν0) ~ -k*Fν[1] + 4*D(Φ)
D(Fν[1]) ~ k/3*(Fν0-2Fν[2]+4Ψ)
[D(Fν[l]) ~ k/(2l+1) * (l*Fν[l-1] - (l+1)*Fν[l+1]) for l in 2:lνmax-1]...
D(Fν[lνmax]) ~ k*Fν[lνmax-1] - (lνmax+1) / τ * Fν[lνmax]
δν ~ Fν0
θν ~ 3k*Fν[1]/4
σν ~ Fν[2]/2
```

Massless neutrino equations in CLASS:

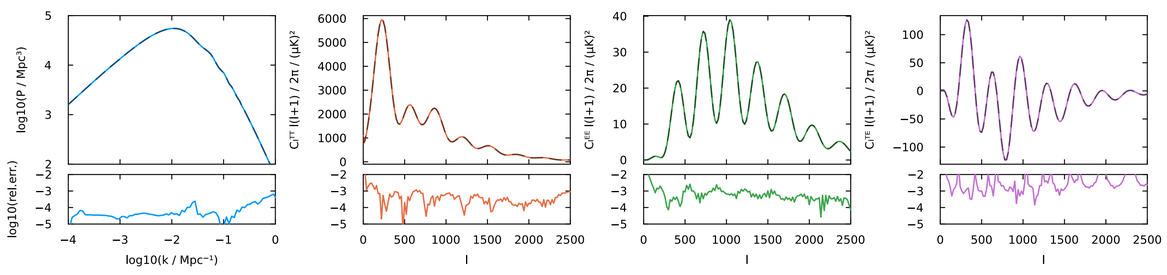
```
/** - ---> ultra-relativistic neutrino/relics (ur) */
if (pba->has_ur == _TRUE_) {
    /** - ----> if radiation streaming approximation is off */
    if (ppw->approx[ppw->index_ap_rsa] == (int)rsa_off) {
    /** - -----> ur density */
    dy[pv->index_pt_delta_ur] =
        // standard term
        -4./3.*(y[pv->index_pt_theta_ur] + metric_continuity)
        // non-standard term, non-zero if ceff2_ur not 1/3
        +(1.-ppt->three_ceff2_ur)*a_prime_over_a*(y[pv->index_pt_delta_ur] + 4.*a_prime_over_a*y[pv->index_pt_theta_ur]/k/k);
    /** - -----> ur velocity */
    dy[pv->index_pt_theta_ur] =
        // standard term with extra coefficient (3 ceff2_ur), normally equal to one
        k2*(ppt->three_ceff2_ur*y[pv->index_pt_delta_ur]/4.-sigma_ur_squared*y[pv->index_pt_shear_ur]) + metric_euler
        // non-standard term, non-zero if ceff2_ur not 1/3
        -(1.-ppt->three_ceff2_ur)*a_prime_over_a*y[pv->index_pt_theta_ur];
    if (ppw->approx[ppw->index_ap_ufa] == (int)ufa_off) {
        /** - -----> exact ur shear */
        dy[pv->index_pt_shear_ur] =
        0.5*(
                // standard term
                8./15.*(y[pv->index_pt_theta_ur]+metric_shear)-3./5.*k*s_l[3]/s_l[2]*y[pv->index_pt_shear_ur+1]
                // non-standard term, non-zero if cvis2_ur not 1/3
                -(1.-ppt->three_cvis2_ur)*(8./15.*(y[pv->index_pt_theta_ur]+metric_shear)));
        /** - -----> exact ur l=3 */
        l = 3;
        dy[pv->index_pt_l3_ur] = k/(2.*l+1.)*
        (l*2.*s_l[l]*s_l[2]*y[pv->index_pt_shear_ur]-(l+1.)*s_l[l+1]*y[pv->index_pt_l3_ur+1]);
        /** - -----> exact ur l>3 */
        for (l = 4; l < pv->l_max_ur; l++) {
        dy[pv->index_pt_delta_ur+l] = k/(2.*l+1)*
            (l*s_l[l]*y[pv->index_pt_delta_ur+l-1]-(l+1.)*s_l[l+1]*y[pv->index_pt_delta_ur+l+1]);
        }
        /** - -----> exact ur lmax_ur */
        l = pv->l_max_ur;
        dy[pv->index_pt_delta_ur+l] =
        k*(s_l[l]*y[pv->index_pt_delta_ur+l-1]-(1.+l)*cotKgen*y[pv->index_pt_delta_ur+l]);
    }
    else {
        /** - -----> in fluid approximation (ufa): only ur shear needed */
        //TBC: curvature?
        /* a la Ma & Bertschinger */
        if (ppr->ur_fluid_approximation == ufa_mb) {
        dy[pv->index_pt_shear_ur] =
            -3./tau*y[pv->index_pt_shear_ur]
            +2./3.*(y[pv->index_pt_theta_ur]+metric_shear);
        }
        /* a la Hu */
        if (ppr->ur_fluid_approximation == ufa_hu) {
        dy[pv->index_pt_shear_ur] =
            -3.*a_prime_over_a*y[pv->index_pt_shear_ur]
            +2./3.*(y[pv->index_pt_theta_ur]+metric_shear);

        }
        /* a la CLASS */
        if (ppr->ur_fluid_approximation == ufa_CLASS) {
        dy[pv->index_pt_shear_ur] =
            -3./tau*y[pv->index_pt_shear_ur]
            +2./3.*(y[pv->index_pt_theta_ur]+metric_ufa_class);

        }
    }
    }
}
```

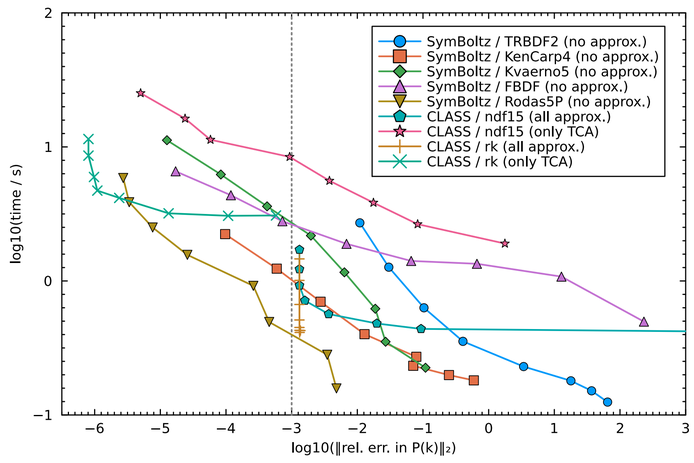
- Similar difference for any part of eqs.
- Reason: symbolic-numeric & no approx.
- SymBoltz: full ΛCDM model in 277 lines.
- CLASS: spread over 27 000 lines & 10 files.

Accuracy: SymBoltz vs. CLASS



- Matter & CMB power spectra agree around $\lesssim 0.1\,\%$ for default-ish precision parameters.

Performance: SymBoltz vs. CLASS



L_2 error of $P(k)$ for perturbation ODE tolerances 10^{-9}-10^{-2} vs. 10^{-10} reference with different ODE solvers. Same k-modes & $\ell_{\max}$. Default massive neutrino sampling. **Lower left (↙) is better.**

- SymBoltz is 10×/1× faster than CLASS with approx. off/on.
- Work in progress to optimize computation of CMB spectra.

Code example

Install and launch `julia`. Load the package and build a standard ΛCDM model:

```
using SymBoltz
M = ΛCDM()
```

This is a *symbolic* representation of all variables and equations in the model. It can be inspected and modified, for example to see the equations in the gravity component "`G`":

```
equations(M.G)
```

$$\frac{\mathrm{d}\,a(\tau)}{\mathrm{d}\tau} = (a(\tau))^2\,\sqrt{\frac{8}{3}\,\pi\,\rho(\tau)} \tag{1}$$

$$\frac{\mathrm{d}}{\mathrm{d}\tau}\,\Phi(\tau,k) = \frac{-k^2\,\Phi(\tau,k)}{3\,\mathscr{H}(\tau)} + \frac{-\frac{4}{3}\,(a(\tau))^2\,\pi\,\delta\rho(\tau,k)}{\mathscr{H}(\tau)} - \mathscr{H}(\tau)\;\Psi(\tau,k) \tag{2}$$

$$k^2\;(\Phi(\tau,k) - \Psi(\tau,k)) = 12\;(a(\tau))^2\;\pi\,\Pi(\tau,k) \tag{3}$$

Now set cosmological parameters and compile the symbolic model to a *numerical* problem:

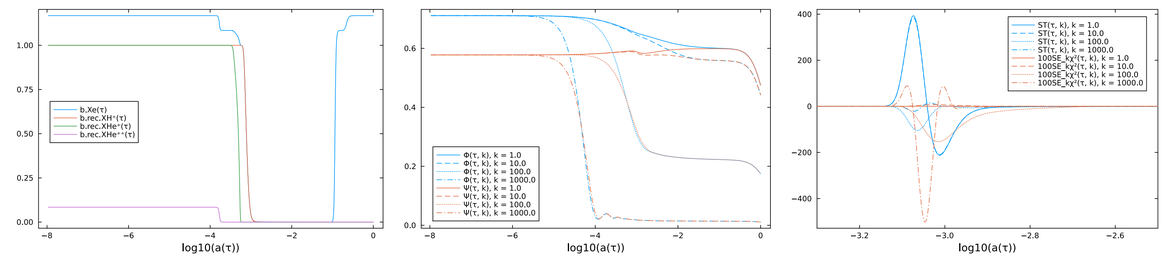
```
p = Dict(
  M.g.h => 0.70, M.γ.T₀ => 2.7, M.c.Ω₀ => 0.27, M.b.Ω₀ => 0.05, M.ν.Neff => 3.0,
  M.b.YHe => 0.25, M.h.m_eV => 0.06, M.I.ns => 0.95, M.I.ln_As1e10 => 3.04
)
prob = CosmologyProblem(M, p)
```

This performs several automatic steps: splitting equations into background & perturbations, splining background variables needed by the perturbations, generating ODE functions & Jacobians, etc. We can solve the background & perturbations for any k-modes:

```
ks = range(1.0, 1000.0, length=500) # k / (H₀/c)
sol = solve(prob, ks)
```

The solution object can easily get or plot *any* symbolic expression as a function of (τ, k):

```
using Plots
ks_plot = [1.0, 10.0, 100.0, 1000.0]
plot(sol, log10(M.g.a), [M.b.Xe, M.b.rec.XH⁺, M.b.rec.XHe⁺, M.b.rec.XHe⁺⁺])
plot(sol, log10(M.g.a), [M.g.Φ, M.g.Ψ], ks_plot)
plot(sol, log10(M.g.a), [M.ST, 100*M.SE_kχ²], [1.0, 10.0, 100.0, 1000.0])
```



Compute matter and CMB power spectra (keyword arguments control precision settings):

```
ks = 10 .^ range(0, 3, length=100)
Ps = spectrum_matter([:m, :cb, :h], prob, ks)

jl = SphericalBesselCache(25:25:2500)
Cls = spectrum_cmb([:TT, :EE, :ψψ], prob, jl)
```

Implement a new $w_0 w_a$ dark energy component (with much less work than in other codes):

```
@parameters w₀ wₐ cₛ² Ω₀ ρ₀=3Ω₀/8π
@variables ρ(τ) P(τ) w(τ) cₐ²(τ) δ(τ,k) θ(τ,k) σ(τ,k)
eqs = [
  w ~ w₀ + wₐ*(1-a)
  ρ ~ ρ₀ * a^(-3(1+w₀+wₐ)) * exp(-3wₐ*(1-a))
  P ~ w * ρ
  cₐ² ~ w - 1/(3ℋ) * D(w)/(1+w)
  D(δ) ~ 3ℋ*(w-cₛ²)*δ - (1+w) * ((1+9(ℋ/k)^2*(cₛ²-cₐ²))*θ - 3*D(Φ))
  D(θ) ~ (3cₛ²-1)*ℋ*θ + k^2*cₛ²*δ/(1+w) + k^2*Ψ
  σ ~ 0
]
initialization_eqs = [
  δ ~ -3//2 * (1+w) * Ψ
  θ ~ 1//2 * (k^2*τ) * Ψ
]
X = System(eqs, τ; initialization_eqs, name = :X)
M = ΛCDM(Λ = X, name = :w₀wₐCDM)
```

Installation and documentation

Easy to install with one command, see:

github.com/hersle/SymBoltz.jl

Feedback is greatly appreciated.
Open an issue to get in touch!

LaTeX TikZposter